

Apple Finder File Edit View Go Window Help

ADBMS_Experiment - 01

Search (Cmd + Ctrl + U)

Home Insert Draw Design Transitions Animations Slide Show Record Review View

Paste Cut Copy Format New Slide Layout Reset Section

Calibri Light (Headings) 54

Convert to SmartArt Picture Shapes Text Box Arrange Quick Styles Shape Fill Shape Outline Add-ins

1 Advance Database Management System (ADBMS)

2 Experiment 01

3 Experiment 01

4 Experiment 01

5 Experiment 01

6 Experiment 01

Click to add notes

Slide 2 of 7 English (United States) Accessibility: Investigate

Notes Comments

104%

Experiment 01

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$a.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table

category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

PowerPoint File Edit View Insert Format Arrange Tools Slide Show Window Help

ADBMS_Experiment - 01

Search (Cmd + Ctrl + U)

Home Insert Draw Design Transitions Animations Slide Show Record Review View

Calibri Light (Headings) 54

Record Share

1 Advance Database Management System (ADBMS)

2 Experiment 01

3 Experiment 01

4 Experiment 01

5 Experiment 01

6 Experiment 01

Click to add notes

Slide 3 of 7 English (United States) Accessibility: Investigate

Notes Comments

104%

Experiment 01

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Output Table:

name	dept_name	salary
Alice	HR	50000
Bob	HR	25000
Charlie	IT	80000
David	IT	40000
Elena	Sales	45000

Servers

Dashboard

Properties

SQL

Statistics

Dependencies

Dependents

Processes

Employee/postgres@PostgreSQL 18

Employee/postgres@PostgreSQL 18

No limit

Query

Query History

```
19 INSERT INTO Departments (dept_id, dept_name) VALUES
20 (10, 'HR'), (20, 'IT'), (30, 'Sales');
21
22 INSERT INTO Employees (emp_id, name, dept_id) VALUES
23 (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
24
25 INSERT INTO Salaries (emp_id, salary) VALUES
26 (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
27 -- 1.
28 select * from Employees e
29 left join Salaries s on e.emp_id=s.emp_id
30 left join Departments d on e.dept_id=d.dept_id;
31
32 -- 2
33 update Salaries set salary=salary+salary*0.1 where emp_id
34 IN (Select emp_id from Employees e join Departments d
35 on e.dept_id=d.dept_id where dept_name='HR');
36
37 DELETE from Salaries where salary<30000;|
38 Select * from Salaries;
39
40 -- 3.
```

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No: 1 of 1

	emp_id [PK] integer	salary integer
1	3	80000
2	4	40000
3	5	45000
4	1	55000

Total rows: 4

Query complete 00:00:03.953

LF

Ln 37, Col 41

PowerPoint File Edit View Insert Format Arrange Tools Slide Show Window Help

ADBMS_Experiment - 01

Search (Cmd + Ctrl + U)

Home Insert Draw Design Transitions Animations Slide Show Record Review View

Calibri Light (Headings) 54 A A A

B I U L X X AV A A

Convert to SmartArt Picture Shapes Text Box Arrange Quick Styles Shape Fill Shape Outline Add-ins

1 Advance Database Management System (ADBMS)

2 Experiment 01

3 Experiment 01

4 Experiment 01

5 Experiment 01

6 Experiment 01

Click to add notes

Slide 5 of 7 English (United States) Accessibility: Investigate

Notes Comments

104%

Experiment 01

(Amazon)

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings. Every customer must choose **exactly 3 different toppings** to prepare a pizza. Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.
The same topping **cannot appear more than once** in a pizza.
Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage
Pepperoni, Chicken, Sausage
Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.
Display the total ingredient cost of each pizza.
Sort the output:
First by **highest total cost**
If two pizzas have the same cost, sort them alphabetically.

